

Machine Learning-Assisted Raman Spectral Fitting Pipeline

Saulo Sampaio Bonfim¹, Eliel Gomes da Silva Neto², Rubisley de Paula Lemes¹

¹Instituto de Computação – Universidade Federal da Bahia (UFBA)

²Instituto de Física – Universidade Federal da Bahia (UFBA)

saulo.sampaio@ufba.br, elielgsn@ufba.br, rlem.rehael@gmail.com

Abstract. *This paper presents a machine learning-assisted pipeline designed to automate Raman spectra deconvolution. To address the challenges of each stage of the pipeline, this study proposes specific algorithms while introducing some purpose-built methods. The experiments to validate the efficacy of the selected algorithms as well as the whole pipeline were performed on synthetic and real spectra. Based on the evaluated test cases the pipeline demonstrated good initial results in curve fitting regarding both accuracy and computational efficiency. We also propose improvements and future work to handle edge cases and more complex spectra.*

Resumo. *Este artigo introduz um pipeline assistido por aprendizado de máquina com o objetivo de automatizar a deconvolução de espectros Raman. O estudo propõe múltiplos algoritmos para resolver os problemas próprios de cada etapa da pipeline e também introduz métodos próprios. Os experimentos foram realizados em bases de dados sintéticas e em exemplos de espectros reais para validar a eficácia dos algoritmos selecionados e do pipeline como um todo. O pipeline mostrou-se eficaz no tratamento dos casos de exemplo tanto em termos de acurácia quanto em custo computacional. Também propomos melhorias e trabalhos futuros para lidar com casos limítrofes e espectros mais complexos.*

1. Introduction

Raman spectroscopy is a powerful non-destructive technique for analysis of materials in a wide range of physical states [Jones et al. 2019, Orlando et al. 2021]. Regarding solid states, the technique has been crucial in the study of nanostructures [Zhang 2012, Ferrari et al. 2006, Gouadec and Colomban 2007]. However, the utility of vibrational spectroscopy is not limited to the domain of solid state physics, there are also many applications in chemistry, food, environment and medicine [Sohn et al. 2020]. The technique has also been used in cancer research, analysis of biofilms, and the development and monitoring of drugs and drug delivery, however, the existence of alternative chemometric techniques may have slowed the widespread adoption of vibrational spectroscopy across various fields. More effective data processing techniques applied to Raman spectroscopy can increase adoption. Therefore, to expand the application of Raman spectroscopy, advanced data processing methods, particularly machine learning algorithms, offer promising solutions for classification tasks [Meza Ramirez et al. 2021].

However, deconvolution of spectra still lacks reliable automated procedures that can reduce operator bias [Lünsdorf and Lünsdorf 2014]. Raman spectroscopy still also

lacks large databases with deconvolution information, and many of the best examples were fitted and deconvoluted using only semi-automated strategies [Mattioda et al. 2024]. The primary objective of this paper is to introduce a custom pipeline of methods designed to efficiently deconvolute large Raman datasets within reasonable computing time. To this end, we selected and evaluated several candidate algorithms, including our own custom methods, based on output quality and runtime performance. This selection process is directly related to the specific functions of each stage of the pipeline, which is initialized through machine learning, thereby forming a machine learning-assisted pipeline. The remainder of this paper is organized as follows: Section 2 covers theoretical foundations, related work, and the proposed methodology; Section 3 addresses the experimental results and discussions; finally, Section 4 provides conclusions and suggests directions for future work.

2. Materials and Methods

2.1. Brief Historical Overview

In an 1899 issue of the *Philosophical Magazine*, John Willian Strutt, also known as Lord Rayleigh from his barony title, proposed that when light travels through a medium such as a gas, the molecules themselves act as emitters of secondary radiation, a phenomenon known as light scattering, or molecular diffraction. He draws from these definitions to explain why the sky appears blue at midday and red during sunset, by showing that the intensity of the scattered light is inversely proportional to the fourth power of its wavelength. In this model, the shorter wavelengths of the sun's light are scattered much more strongly and thus deviate from the path of the primary wave, leaving the longer wavelengths scattered along the longer path of the atmosphere, causing the red sunset for more distant observers; this elastic scattering of light is today known as Rayleigh scattering [Rayleigh 1899].

Sir Chandrasekhara Venkata Raman discussed Rayleigh's work in his 1922 book titled *Molecular Diffraction of Light*, where he posits this question on the first chapter: "does any departure from perfect regularity of the light-propagation arise from the discontinuous structure of the medium?" [Raman 1922]. This inquiry laid the groundwork for Raman to discover inelastic scattering associated with the "internal properties" of the medium, as he described, affect scattered light. In his later 1928 publication on *Nature*, Raman announced his discovery of a new type of secondary radiation, which differed from Rayleigh's scattered light by presenting a shift in wavelength from the incident primary radiation [Raman and Krishnan 1928]. This rather subtle effect, now known as Raman Scattering, occurs in approximately only one out of every 10^6 to 10^8 scattered photons [Smith and Dent 2019]. This pioneering work, alongside his student K.S. Krishnan, earned Raman the Nobel Prize in Physics in 1930, making him the first Asian non-white person to receive the award. Krishnan was also subsequently recognized for his vital contributions.

The Raman effect, as it is known today, was originally observed through an apparatus consisting of a telescope, a sample chamber, and two filters. These filters were selected such that the incident light would be totally extinguished when they were coupled. In the experiment, sunlight passed through a violet filter before illuminating a liquid or gas sample. While the majority of the light was scattered with the same frequency of

the incident beam, which corresponds to Rayleigh scattering, a small fraction was scattered at a different frequency. To observe this inelastic effect it is necessary to insert a filter to block excitation light and the Rayleigh scattered light [da Silva Neto 2017].

2.2. The Raman Effect and Spectroscopy

From a modern perspective, the Raman effect is described as the resulting interaction between incident photons and matter. When interacting with a medium, a photon's energy may be absorbed, scattered, or pass through without interaction. When light interacts with the electron cloud, it produces a temporary excited state which is called a virtual state because it exists only for an instant. The molecule quickly collapses from that virtual state and scatters a photon [Smith and Dent 2019].

The instability and short life of the virtual state is why the language around scattering is similar to the language around collisions. If only the electron cloud was distorted during the process, the scattered light will have the same frequency as the incident light which corresponds to Rayleigh scattering. If, however, the vibrational state is also affected, the material may absorb the energy from the photon, causing it to be scattered at a lower frequency or, conversely, the photon may absorb energy from the molecule and scatter with a higher frequency, which is the Raman effect [Smith and Dent 2019].

Since the primary objective in vibrational spectroscopy, such as Raman spectroscopy, is to analyze molecular vibrational states, the spectrometer output is typically expressed in terms of energy rather than wavelength. We can then use the wavenumber ϖ , which is directly proportional to energy as shown in Equation Equation 1, and is typically measured in cm^{-1} . We call this energy difference the Raman Shift [Smith and Dent 2019].

$$\varpi = \frac{\nu}{c} = \frac{1}{\lambda} \quad (1)$$

By measuring the wavenumber of the scattered photons, a spectrum graph is constructed with the wavenumber (ϖ) on the x-axis and the relative intensity of the scattered light on the y-axis. Intensity is typically represented in arbitrary units, but it is directly related to the distribution and wavenumber of the photons that reach the spectrometer's sensors [Smith and Dent 2019].

Since most photons are scattered elastically, the spectrum graph will show a strong concentration of photons around the point $x = 0 \text{ cm}^{-1}$, which means no Raman shift has occurred. This band is commonly excluded from Raman graphs, since it does not correspond to Raman scattering. Raman scattering itself has two modes and they are called Stokes or anti-Stokes. In Stokes scattering, the scattered light has less energy than the incident light, and thus those photons will be represented in the negative portion of the spectrum graph. In anti-Stokes scattering, the scattered photons have gained energy from the molecules and have more energy than the incident photons, and thus are distributed in the positive portions of the spectrum graph [Smith and Dent 2019].

2.3. Modeling Raman Spectra

The underlying phenomenon behind Raman spectral data is the oscillation of electrons disturbed by electromagnetic waves. A standard mathematical model for this oscillation

behavior is the forced damped harmonic oscillator, where oscillations are driven by external forces induced by the varying electric field and modulated by radiation damping [Griffiths 2013]. From a classical mechanics perspective, the forced damped harmonic oscillator can be modeled as a mass m on a spring system with friction b driven by an external force F_0 applied at a certain frequency ω . The classic equation for this oscillator model is shown by Equation 2.

$$m\ddot{x}(t) + b\dot{x}(t) + kx(t) = F_0 \cos(\omega t). \quad (2)$$

Modeling the intensity spectrum $A(\omega)^2$ of this motion along an axis as a function of the frequency of the external driving force yields the proportionality relationship shown in Equation 3a, which also applies to the modeling of a molecule as a vibrating system. The variable ω represents the frequency of the driving force, Δ is the energy difference between the driving frequency and the fundamental frequency, also called resonant frequency, of the molecule, as shown in Equation 3b, and γ is the full width at half maximum of the signal band.

$$A(\omega)^2 \propto \frac{1}{\Delta^2 + \left(\frac{\gamma}{2}\right)^2} \quad (3a)$$

$$\Delta = \omega - \omega_0 \quad (3b)$$

This function is a Cauchy distribution, also known as a Lorentzian curve. Indeed, given the correct normalization, each peak in a Raman spectrum can be modeled as a Lorentzian peak and the complete spectrum function expressed as a summation of these individual peaks. The process of decomposing the spectrum back into its peaks and their defining parameters is known as deconvolution.

This model enables not only the deconvolution of real spectra but also the generation of synthetic datasets with normally distributed noise. To this end, a function can be utilized to construct synthetic spectra with profiles similar to the real data and it is also the global fitting function for Raman deconvolution. We discuss this further in Section 2.6.

2.4. Automated Analysis of Raman Spectra

When data are applied to substance identification tasks, models that compute the proximity between a tested sample and previously labeled dataset samples offer substantial utility. To this end, significant performance has been achieved using convolutional neural networks (CNNs). This efficacy relies on the fact that materials maintain a unique and approximately invariant spectral signature, regardless of its sample source [Mozaffari and Tay 2022].

In another example, where the focus was the effects of external phenomena on a material, such as the impact of UV exposure on biological samples, obtaining specific information regarding how the spectral signature shifts in response to the analyzed external variable is critical for accurate analysis [Sohn et al. 2020].

The most comprehensive approach to Raman data analysis involves the full deconvolution of the spectrum into its component peaks (also referred to as features or Raman bands) and their respective parameters using a Voigt function or, more specifically, Cauchy-Lorentz curves. This task is executed through a combination of automated and manual techniques, culminating in a local least-squares optimizer, such as the Levenberg-Marquardt algorithm, to extract the parameters that describe each peak. Thus, this solution represents the most challenging to achieve through a fully automated procedure.

Applications such as Fityk provide a diverse toolkit for spectroscopists to perform deconvolution of Raman signals, but they still require manual adjustments in difficult spectra, and the spectroscopist manually determines the number of curve fitting iterations to be performed [Wojdyr 2010].

2.5. Related Works

In a 2014 conference paper, the authors proposed an automated Raman spectra fitting method based on iteratively adding Lorentzian peaks and performing fitting with randomized initial parameters to reduce the sum of squared residuals (SSR). This approach can be described as a brute force solution, which faces challenges in computational cost for large datasets, due to it repeatedly performing curve fitting on an increasing number of fit parameters. This approach is further limited by its stopping criterion, which can lead to overfitting, reducing the SSR while producing confusing information about non-existent features. This solution may be useful when fitting a small set of spectra with manual intervention. The authors also proposed that the way to handle with overlapping peaks is to repeatedly fit them to observe the ambiguity of the band being studied [Lünsdorf and Lünsdorf 2014].

The AMES Research Center from NASA maintains high quality databases on astrophysics and also vibrational spectroscopy. Their Raman Spectroscopy database (*Ramdb*) had features labeled using a semi-automated system described in their 2024 article [Mattioda et al. 2024]. They utilized the peak finding function from the Scipy python library, which performs a direct comparison of neighboring data points, to establish an initial estimate of the number and positions of spectral features in the Raman signal [Virtanen et al. 2020]. These estimates were subsequently used as initial parameters for a local optimizer. However, at the conclusion of that stage and all subsequent stages, visual inspection by experts was necessary to evaluate the quality of the produced fits and perform manual adjustments.

The primary objective of this article is to develop a methodology to initiate global optimization on spectra without the need for manual peak counting. Global optimizers are inherently more suitable for automated curve fitting, due to local optimizers requiring precise initial values, while global methods rely only on predefined ranges of possible parameter values. There are several well established global optimizer functions that were evaluated in this study as candidate solutions, namely Differential Evolution (DE) [Storn and Price 1997], Dual Annealing (DA) [Righini 1995], and the more recent Simplicial Homology Global Optimization (SHGO) [Endres et al. 2018]. In this study a custom global optimizer based on the K-Nearest-Neighbors algorithm was also developed, which combines synthetic curve generation, grid refinement of parameter search spaces, and finalization via Levenberg-Marquardt (a local optimizer). SHGO was excluded from the final results due to its poor performance across multiple test scenarios.

2.6. Proposed Approach

2.6.1. The Spectra

Since the objective is to develop an automated pipeline to perform deconvolution of arbitrary Raman spectra with minimal instance specific parameters, this study utilizes synthetic spectra constructed as summations of Cauchy-Lorentz distributions. These distributions are normalized to have their peak amplitudes at a parameterized value A . Thus, the function used is not in its probability density function form.

$$F(x) = \sum_{i=1}^n \frac{A_i}{1 + \left(\frac{(x-x_{0i})}{\gamma_i}\right)^2} \quad (4)$$

Equation 4 shows the function to produce a synthetic spectrum with n Raman peaks. Parameters A_i , γ_i and x_{0i} represent, respectively, the amplitude, half-width at half-maximum (HWHM), and position of a peak i . The parameters for each peak are randomly generated using a uniform distribution function across scaled parameter ranges to maintain consistency.

This approach enables the randomization of feature parameters to construct large datasets of generic spectra. Synthetic datasets also ensure that the ground truth for deconvolution remains available, even for exceptionally large data volumes. Although public Raman datasets containing real samples are available, they often have small sample sizes, lack annotations and deconvolution information, or have a limited variety of spectral signatures to train machine learning models, factors that can compromise the generality we aim to achieve with this pipeline. To validate the pipeline under real-world conditions, a subset of the AMES database was utilized as a test suite.

Incorporating noise is a crucial component in constructing synthetic Raman datasets. In this context, white noise is commonly modeled as a zero-mean Gaussian distribution, with the corresponding probability density function detailed in Equation 5.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (5)$$

The standard deviation (σ) of the noise is used to regulate the noise intensity in this study. Normally, under empirical laboratory conditions, noise is typically reported as the relationship between signal power and noise power, known as the signal-to-noise ratio (SNR). The noise σ can be related to the SNR by dividing the signal power, represented by μ^2 in Equation 6a, by the desired target SNR and extracting the square root, as expressed in 6b.

$$\mu^2 = \frac{\sum_{i=1}^n y_i^2}{n} \quad (6a)$$

$$\sigma = \sqrt{\frac{\mu^2}{\text{SNR}}} \quad (6b)$$

From these equations, it is possible to see that a truly noiseless curve can not be produced using this SNR definition. Therefore, in this study the noise in a dataset is controlled directly using the standard deviation, and when relevant, the use of SNR will be reported. But, it is important to note that there are some divergences in the literature regarding SNR definitions. In some frameworks, the SNR is defined such that it approaches 0 (zero) as noise increases relative to the signal features. In alternative formulations, the SNR approaches 1, which is the case of the definition adopted in this study.

An underlying assumption behind the proposed Raman spectra modeling is that baseline correction has been previously executed. Given the lack of standard methodologies for baseline correction, it is expected minor discrepancies between the fitted peaks and the spectral features around the tail area of the peaks when applying the model to real spectra from open datasets.

2.6.2. The Pipeline

Spectral deconvolution is a difficult task, and the methods involved often require a human in the loop. The task involves identifying Raman peaks and fitting the curve with a Voigt profile curve, which is a convolution of Cauchy-Lorentz and Gaussian curves. Cauchy-Lorentz distributions are solutions to a forced damped harmonic oscillator, thus it's more appropriate to deconvolute the spectrum using this function.

The most widely used curve fitting approach is Levenberg-Marquardt algorithm (LM) for solving least-squares optimization, which is often the default method in signal processing libraries such as LMFit. This Python library also provides alternative optimization methods. However, the local optimizers suffer from a critical limitation that is their dependence on accurate initial parameter estimations to successfully converge on a solution, hopefully toward the global minimum.

The human in the loop remains necessary to bridge the gap between the peak identification task and parameter initialization. This process becomes particularly challenging when peaks overlap and are difficult to identify, as typically observed in the D and G bands that appear in the Raman spectra of carbon materials such as graphene and graphite.

In this context, this study proposes a pipeline to perform Raman deconvolution with three stages: peak identification and counting, parameter initialization, and fitting error minimization. This proposed architecture is designed to operate with minimal required assumptions about the spectra, ensuring that any assumptions applied must be globally applicable to entire datasets rather than restricted to particular instances.

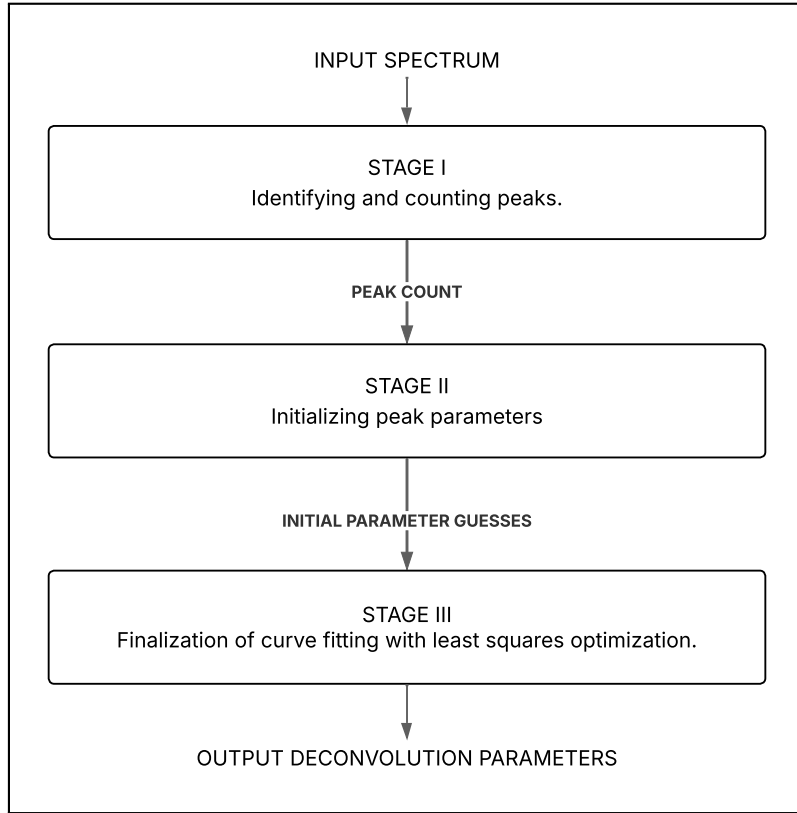


Figure 1. Proposed pipeline

In each stage of this pipeline, different models were used to perform the appropriate task to that stage. For stage I, we primarily tested a multi-kernel convolutional neural network, which is a modified version of the one proposed by [Sohn et al. 2020]. We also tested Random Forest (RF) and compared the results of these two approaches. The goal of stage I is the classification of spectra according to their peak counts. The output of this stage are the 3 most likely peak count probabilities.

For stage II, a K-Nearest Neighbors (KNN) inspired approach, which we decided to call Adaptive kNN (A-kNN), was developed to procedurally simulate spectra and iteratively reduce the parameter search space for neighbors. This approach does not require a reference dataset, since it generates spectral profiles dynamically, storing only the closest neighbors in memory via a Max Heap structure.

The iterative refinement of the parameter search space ensures convergence to a minimum. Within this architecture, the sample generation size is a key hyperparameter, given a sufficient sample volume, A-KNN can achieve successful curve fitting across numerous scenarios. However a trade-off can be established by using fewer samples to reduce computational cost, yielding approximate neighbors that do not provide a perfect fit but are subsequently refined during Stage III.

This approach optimizes the brute-force method applied by [Lünsdorf and Lünsdorf 2014], as it requires least-squares curve fitting to be only performed on each neighbor found once, thus a total of k times. Furthermore, generating random synthetic spectra and computing the p-norm is also substantially less computa-

tionally expensive. In this implementation, the three peak count probabilities returned by Stage I are processed in parallel to accelerate execution.

Finally, Stage III uses the output from Stage II to perform LM fitting on the spectra, employing the parameters of the nearest neighbors as initial guesses. This stage is also executed in parallel. The resulting profiles are subsequently processed by the parent process, which applies the Bayesian Information Criterion score to return the simplest and most accurate best-fitting curve required to resolve the spectrum.

2.6.3. The Models

The original Single Layer M model proposed by [Sohn et al. 2020] was utilized to classify biological Raman spectra across three classes corresponding to the UV radiation exposure times of skin samples from a mid-sized pig. This neural network was distinguished by its unique structure, which featured a single stage with multiple convolution kernels of different sizes. The objective of this architecture is to capture Raman features with different spectral widths, thereby preventing nearby narrow peaks from being convoluted together and avoiding the loss of relevant features. Given that such spectral distortions would affect the accuracy of the peak-counting task in Stage I, the multi-kernel SLMK-CNN architecture was selected as a core component of this pipeline.

To address the increased complexity of this task, several modifications were introduced to improve the original accuracy of the results achieved during the first trials using the baseline network. These enhancements were implemented incrementally by retraining and evaluating the model on synthetic datasets, always using isolated training and test sets.

The SLMK-CNN was extended by incorporating a batch normalization layer after each convolution kernel to enhance model stability against highly diverse spectral signatures. These layers are followed by a ReLU activation and a pooling layer. The architecture then passes through a flattening layer, similar to the original model. Furthermore, additional fully connected layers were integrated to progressively reduce the dense feature space between the flattening layer and the final output layer. In this configuration, the output layer comprises five outputs, given that the peak counts in the database range from 1 to 5. The architecture is detailed in Figure 2.

CNN ARCHITECTURE

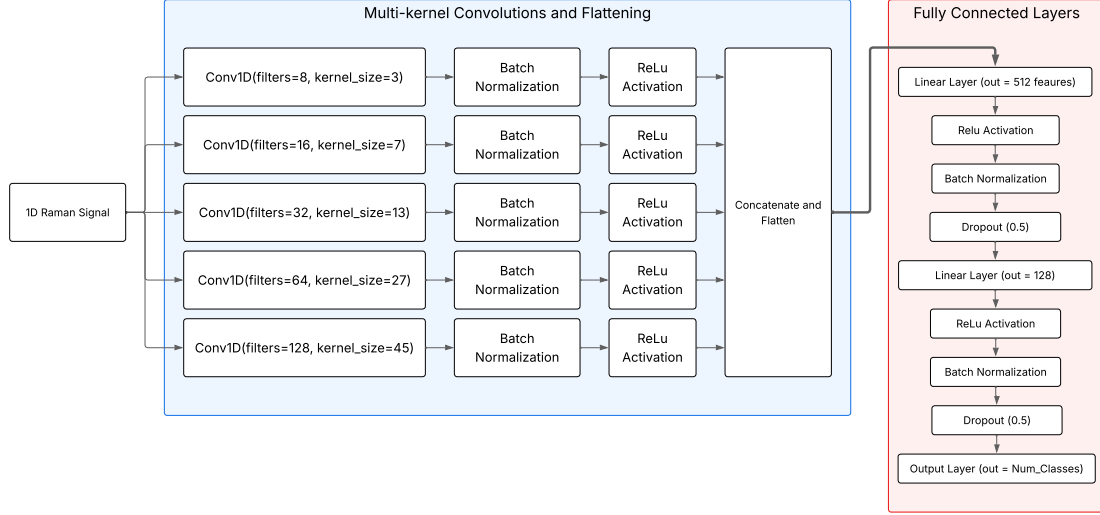


Figure 2. Architecture of Neural Network

A Random Forest model was utilized as a benchmark for the proposed neural network, configured with an ensemble size of 100 decision trees. Gini impurity was selected to measure feature importance, as Log-loss was found to yield equivalent performance compared to Gini, consistent with documented benchmarks [Virtanen et al. 2020]. The model was evaluated using both the raw spectral data and a shifted version of the spectra, where the signal is shifted along the y-axis to count the number of times multiple horizontal lines at specific sampling thresholds intersect the spectrum. This spectral transformation, executed by shifting the axis, significantly enhanced Random Forest classification accuracy during preliminary tests. Consequently, this approach was standardized for the evaluations presented in the Experimental Results Section.

The objective of Stage II is to compute suitable initial fitting parameters based on the peak counts generated in Stage I. For this reason, a modified version of the K-NN algorithm was implemented, which operates as a hybrid approach integrating conventional K-NN framework with a search algorithm based on simulations. The performance of this approach was evaluated against robust global optimizers, specifically Differential Evolution (DE) and Dual Annealing (DA).

The procedure detailed in Algorithm 1 generates a parameterized number of randomized, noiseless spectral samples, with their parameters constrained within global ranges. Subsequently, the Euclidean distance between each generated sample and the noisy target spectrum is computed. The k nearest neighbors of the target spectrum are stored in a max-heap. At the conclusion of the sampling process, a subsequent refinement iteration is initiated after the peaks from these neighbors are clustered by parameter proximity. If a cluster size is greater than half the total number of neighbors, indicating that a majority of the neighbors align with the same peak, then the parameter ranges for that peak are updated to reflect the minimum and maximum values within that cluster.

Algorithm 1 Adaptive K-Nearest-Neighbors

```
1: procedure FITSPECTRUM
  Input:
     $S$  : target spectrum
     $n$  : number of peaks
     $p$  : samples per round
     $r$  : number of rounds
     $k$  : size of nearest-neighbor heap
  Output:
    Set of  $k$  nearest-neighbor solutions

2:    $\mathcal{P} \leftarrow$  Construct peak-center probability map from  $S$ 
3:    $H \leftarrow$  empty max-heap of capacity  $k$ 
4:    $R \leftarrow$  initial peak parameter ranges
5:   for  $t = 1$  to  $r$  do
6:     for  $i = 1$  to  $p$  do
7:       if  $t = 1$  then
8:         Sample peak centers from probability map  $\mathcal{P}$ 
9:       else
10:        Sample peak centers from current center ranges in  $R$ 
11:      end if
12:      Sample amplitudes and widths from ranges in  $R$ 
13:      Generate synthetic spectrum  $\hat{S}$ 
14:      Normalize  $\hat{S}$ 
15:      Compute Euclidean distance between  $S$  and  $\hat{S}$ 
16:      Update heap of  $k$  nearest neighbors
17:    end for
18:     $\mathcal{C} \leftarrow$  Peak clusters by parameter proximity
19:    for all  $c \in \mathcal{C}$  do
20:      if Size of  $c > k/2$  then
21:        Restrict peak parameter ranges using cluster extrema
22:      end if
23:    end for
24:  end for
25:  return All neighbors in the heap  $H$ 
26: end procedure
```

The procedure returns the k nearest neighbors relative to the target spectrum. This process is computationally efficient because it requires no analytical model to generate successive samples and does not execute expensive iterative fits. The computational cost scales linearly with the number of samples, number of rounds, and peaks being fitted, assuming all other parameters remain constant.

The algorithm performs robustly against noisy spectra, but it may introduce errors when fitting spectra with highly disparate peak amplitudes, specifically, when one or more peaks have dominant amplitude while the others are comparatively small but yet distin-

guishable from the noise baseline. This limitation is due to the fact that Euclidean distance assigns greater weight to intense features than to weaker ones. Consequently, capturing very small peaks would require adjusting the global parameter settings to include lower amplitude values, which expands the search space, reducing the sampling probability of generating a proper neighbor that incorporates the minor peaks in question.

In scenarios where the relative amplitude of the peaks is high, meaning that the smaller peaks maintain a sufficient SNR to remain visible, and all peaks are easily separable, a greedy approach can yield superior performance at a minimal computational cost. This alternative framework is discussed further in the Future Work section.

Stage III utilizes a group of closely related spectral profiles and their curve parameters returned by A-kNN to initialize the Levenberg-Marquardt algorithm, which performs the parameter refinement. All k nearest neighbors relative to the target spectrum are refined via LM optimization, and the resulting profile yielding the lowest χ value is selected as the optimal fit for that specific peak count. This optimization workflow is executed in parallel across the three candidate peak counts generated during Stage I.

Stage III also selects the optimal model from the three parallel processes based on the three different peak counts, utilizing the Bayesian Information Criterion (BIC). The LMFit implementation of this formulation, as expressed in Equation 7 [Newville et al. 2025], was utilized for this calculation. The lowest BIC score identifies the simplest model that best fits the data, which is then returned as the final fit to the target spectrum. The equation takes into account the number of data points N , the number of variable parameters N_{vars} , and the χ^2 error.

$$\text{bic} = \ln\left(\frac{\chi}{N}\right)N + \ln(N)N_{\text{vars}} \quad (7)$$

3. Experimental Results and Discussion

The models were initially evaluated independently to measure their capacity to produce the expected output at each specific stage of the pipeline. Subsequently, the entire framework was evaluated on the end-to-end performance of automated curve fitting.

3.1. Stage I Models

The Random Forest classifier and the proposed CNN were compared based on their performance in counting spectral peaks. For this evaluation, a training dataset containing 50,000 randomized synthetic Raman spectra was generated, alongside a separate validation set of 10,000 testing samples. The objective of counting peaks was modeled as a classification task rather than a regression problem. Preliminary testing demonstrated that the neural network achieves better performance when evaluating the outputs as discrete classes.

Both the training and test sets contained between one and five peaks and were completely balanced across these categories. Regarding noise distribution, the training dataset was partitioned into three distinct groups: low noise ($0.01 \leq \sigma \leq 0.1$), medium noise ($0.1 < \sigma \leq 0.25$), and high noise ($0.25 < \sigma \leq 0.5$). Because the maximum peak amplitude was normalized to 1.0, 95% of the generated noise points at the highest noise intensity could reach values up to the maximum peak amplitude itself.

The noise levels were intentionally imbalanced in the training dataset to prioritize lower noise samples. The final configuration allocated 70% of the training set to low noise, 20% to medium noise, and 10% to high noise. Within each of these groups, the noise standard deviation was approximately uniformly distributed utilizing the NumPy random number generator [Harris et al. 2020]. In contrast, the testing dataset incorporated a uniform noise distribution across its entire range, introducing a proportionally higher concentration of high-noise samples during evaluation compared to the training phase.

Defining realistic curve parameters for randomized synthetic spectra dataset is challenging, even under normalization and scaling assumptions. This difficulty is especially sensitive when determining the lower bound value for amplitude, as well as both the lower and upper bounds values for HWHM.

The entire dataset was mapped within a normalized Raman Shift range between 0 and 1 arbitrary units, utilizing a resolution of 1,000 data points. The maximum peak amplitude was set to 1.0, whereas the lower bound for amplitude was established at 0.01 units, meaning that the smallest possible simulated peak is 100 times smaller than the dominant peak. Finally, the HWHM range was constrained between 0.003 and 0.05, which correspond to the size of the smallest convolution kernel within the neural network and 5% of the total spectral window size, respectively. To maintain consistency across all profiles, peak heights were scaled to ensure that the maximum intensity point always has the value 1.0.

These constraints are intentionally broad given the diverse profiles of real Raman spectra. Incorporating more specific and realistic parameter boundaries should achieve better accuracy in all curve fitting tasks within the pipeline.

3.1.1. The Neural Network

In the first round of tests, an independently generated test dataset was utilized based on the same parameter constraints as the training set, but featuring a uniform distribution across noise levels. In terms of classification performance, the model achieved superior overall accuracy compared to the Random Forest approach. During both the training data-loading phase and evaluation, a Savitzky-Golay filter was applied to the spectra with a window length of 21 data points and a polynomial order of 3. These filter parameters were found to consistently provide effective denoising across the samples.

Some exploratory analysis of the model were performed using k-fold with 5 folds to verify model stability, each fold being trained on 15 epochs, and the best validation accuracy being analyzed. The model achieved validation accuracy between 53% and 54% in all folds during the analysis phase.

Finally, the neural network was trained over 25 epochs on all 50,000 training instances and the optimal weights from the 24th epoch were selected. The model was then evaluated on a test dataset containing 10,000 samples, generated under identical baseline conditions, resulting in a dataset contained 2,000 samples per class.

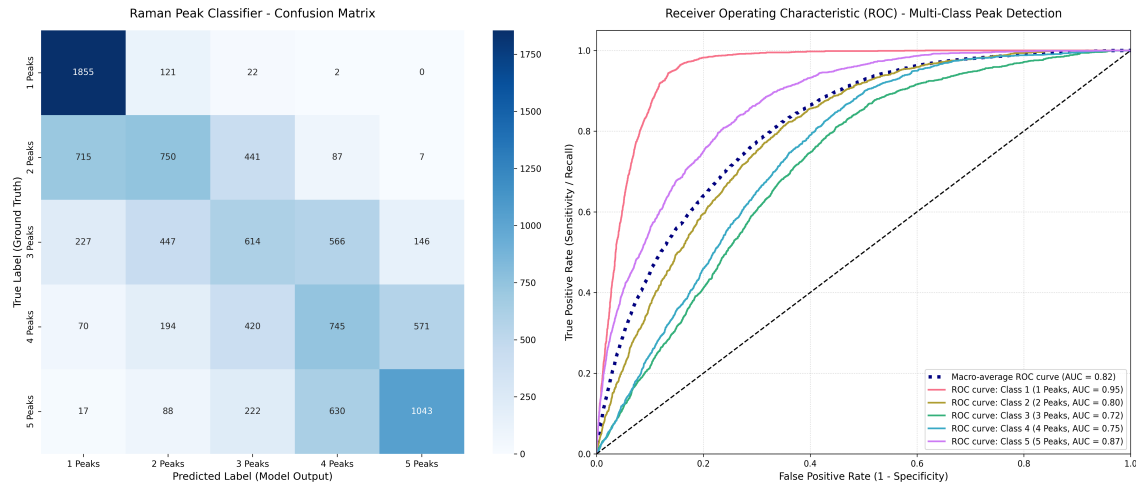


Figure 3. Number of peaks estimation results as a classification problem. The graph on the right shows the ROC curves for each class in the framework, as well as the macro-average for the whole test set.

Figure 3 demonstrates that the neural network is capable of learning complex spectral features. The total classification accuracy achieved was 50.07%. During the training phase, KL-Divergence loss was utilized to attribute weights to the inferred values. Because underestimating the number of peaks was deemed preferable to overestimation, a weight of 0.8 was assigned to the true labels and 0.2 to the -1 error labels. Despite this intentional offset, the network still presents a tendency to overestimate rather than underestimate results in certain scenarios. Nevertheless, within the acceptable tolerance boundary of a -1 error, the network achieved an accuracy of 72.19%.

The core challenge within this framework lies in detecting individual peaks within a spectral superposition characterized by highly similar parameters, which causes them to appear as a single stronger peak. The presence of residual noise, even after smoothing via the Savitsky-Golay filter, further complicates detection in scenarios where these overlapping peaks should remain distinguishable, such as when one peak exhibits a significantly lower amplitude compared to its overlapping pair.

3.1.2. Random Forest

The Random Forest classifier demonstrated lower accuracy compared to the neural network when evaluated on the same training and test datasets, achieving an overall accuracy of 40.57% across all classes. These results, however, are somewhat nuanced, and Random Forest remains a viable alternative for integration into the machine learning stage of the pipeline. To clarify the RF utilization, it is necessary to examine the limitations of the proposed neural network.

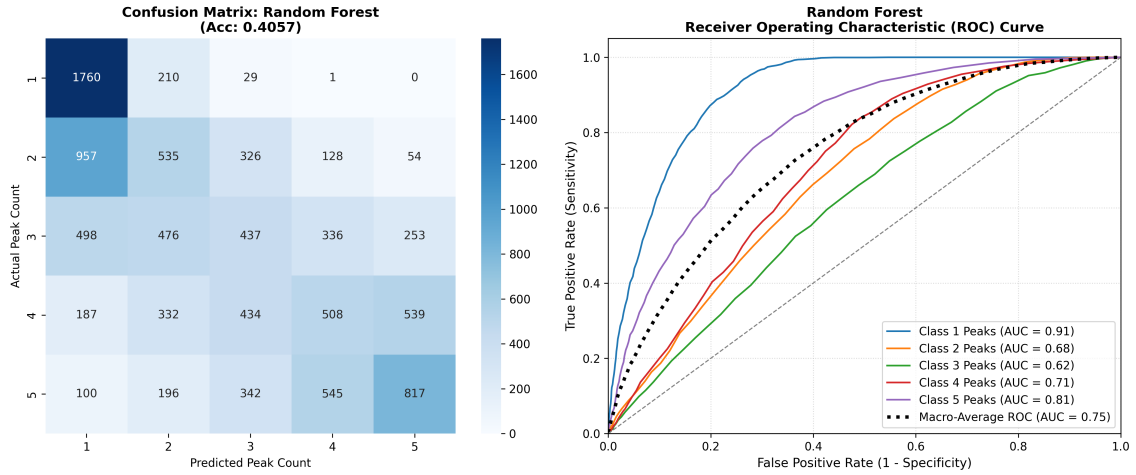


Figure 4. Confusion matrix and ROC for Random Forest. While the neural network achieved greater accuracy overall, Random Forest also converges on the right labels and it may provide some other advantages.

The core limitation of the proposed machine learning component within this pipeline is its rigidity compared to the algorithms utilized in Stages II and III, specifically, it can only generalize within the definition boundaries of its training dataset. Consequently, handling datasets with alternative boundaries and parameters requires complete model retraining. For the neural network, this retraining phase is computationally expensive, necessitating a Graphics Processing Unit (GPU) to complete within a reasonable timeframe. In contrast, the Random Forest model is highly optimized for the CPU and can train on large datasets rapidly, requiring only a few minutes, while the neural network, which is relatively shallow, requires a few hours to train using the Google Colab’s T4 model TPUs.

Additionally, the classification performance of the Random Forest model can be evaluated from a different perspective. When considering the top three classifications based on the model by confidence, the correct label was included within the candidate set in 87.02% of cases. Because the proposed pipeline analyzes these top three predictions from the model, utilizing the Random Forest classifier ensures that an instance with the correct peak count is processed in the vast majority of trials, better than randomly predicting the true peak count. This characteristic, combined with its operational flexibility, makes Random Forest an attractive model for the pipeline. In this study, the neural network was utilized as a proof of concept and performance baseline. However, future versions of the framework will thoroughly evaluate the Random Forest as a potential replacement, as discussed in greater depth in the Future Work section.

3.2. Stage II Methods

The proposed approach for curve fitting, which utilizes a kNN based algorithm to generate approximations of the target curve prior to local refinement via the Levenberg-Marquadt (LM) algorithm, has proven highly effective in terms of both accuracy and execution time. This framework was evaluated alongside well known global optimizers methods across an initial test suite consisting 180 synthetic curves containing between one and three peaks. To assess noise robustness, the SNR values were varied among 5, 20 and 100. The test

spectra featured a resolution of 200 data points. Computational runtime, the number of objective function evaluations, and overall optimization success rate were evaluated for Differential Evolution (DE), Dual Annealing (DA) and the proposed Adaptive K-Nearest Neighbors (A-kNN). Equation 8 defines the criterion used to determine fit success. For the A-kNN implementation, hyperparameter configurations were established at $k = 5$ neighbors, 4 refinement iterations with 1000 samples per round, and a parameter error tolerance of 0.15 for peak clustering.

$$\chi^2 \leq ||y_{\text{noisy}} - y_{\text{clean}}||^2 + \epsilon \quad (8)$$

The term ϵ represents an error threshold added to the baseline error introduced by noise. Table 1 details the results of the three benchmarks under a highly permissive threshold constraint. While all evaluated approaches demonstrated similar performance regarding optimization success rates, the proposed A-kNN combined with LM achieved significantly lower runtimes compared to all other methods in the benchmark.

Table 1. Benchmark ($\epsilon = 10^{-3}$)

Method	Avg NFE	Avg Runtime (seconds)	Success Rate (%)
Differential Evolution	5686	1.0517	97.22
Dual Annealing	12706	1.7388	96.67
Adaptive kNN + LM refinement	5555	0.1968	99.44

An interesting detail about the A-kNN combined with LM is the highly similar number of objective function evaluations compared to Differential Evolution, but demonstrating superior computational efficiency. This performance can be attributed to the lower computational complexity of the operations executed during a typical A-kNN iteration compared to overhead of DE and DA algorithms. While A-kNN randomly generates samples and evaluates proximity using highly efficient vectorized Euclidean distance, DE must compute successive samples using formulae [Storn and Price 1997].

Another operational advantage of the A-kNN framework is the independence of its generated samples, which may slightly affect initial fit quality, but mitigates the risk of the algorithm becoming trapped in local minima. Therefore, the complexity of the algorithm is directly related to the expected output quality, providing a highly customizable and transparent trade-off between execution runtime and fitting accuracy.

The structural limitations of DE and DA became evident during the benchmark, because the problem set was initially designed to include synthetic spectra with five peaks and these traditional algorithms were unable to converge on successful fits within an acceptable execution timeframe. In contrast, the A-kNN method demonstrated robust scalability due to its core computational workload scaling linearly with the optimization dimensionality, ensuring that the primary computational overhead is strictly bounded by the execution of the local optimizer.

3.2.1. Experiments on Real Spectra

A second test suite was established to benchmark the algorithms using experimental spectra obtained from the AMES Raman Database, maintained by the Astrophysics and Astrochemistry Laboratory at NASA’s Ames Research Center [Mattioda et al. 2024]. A total of ten samples, incorporating distinct segments of identical spectra in specific cases, were selected from solid crystals and microcrystals signatures. This round of experiments assessed the capacity of each algorithm to converge on previously annotated spectral features. Within this experimental setup, all algorithms were provided with the exact number of target peaks. Operational configurations for the A-kNN framework were maintained at 4 refinement rounds involving 1,000 samples each, with $k = 5$ nearest neighbors.

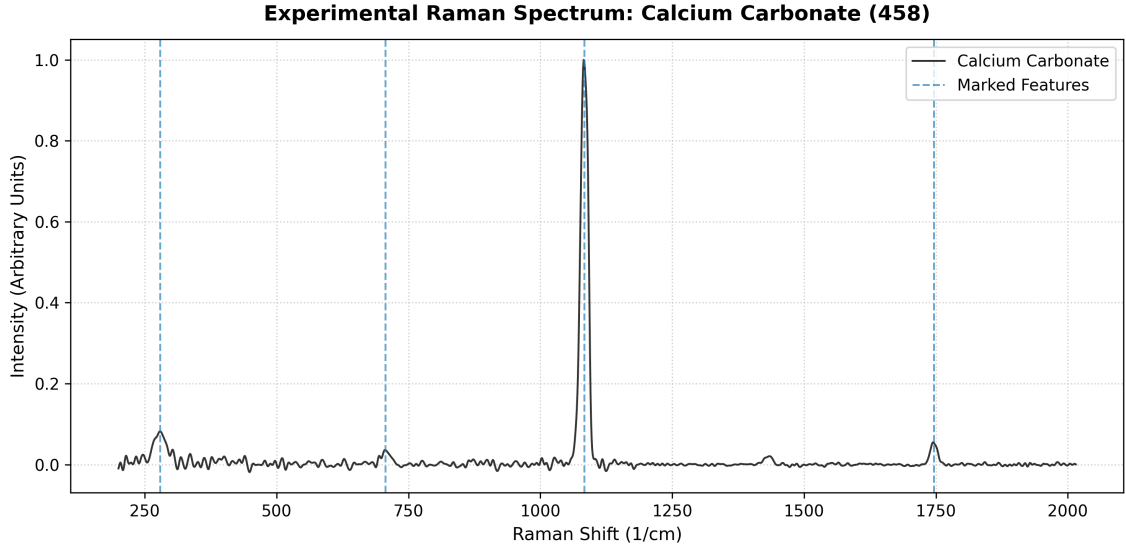


Figure 5. Calcium Carbonate cropped at the spectral window between 200 and 2016 cm^{-1} . The blue vertical lines show the positions of the features labeled by the maintainers of the database. Although the peak at 1083 cm^{-1} is easily distinguishable, the other features are harder to fit automatically, specifically the ones at the Raman bands of 271 cm^{-1} , 706 cm^{-1} and 1746 cm^{-1} .

The data were normalized to maintain a consistent spectral window bounded between 0 and 1, applying the same preprocessing approach utilized to evaluate the training and test synthetic data on Section 3.1.1. Normalization along the y-axis was performed directly by the authors. Given that this database was fitted using a semi-automated process, as described by the original authors, minor variations may exist concerning the classification of annotated features as true Raman peaks.

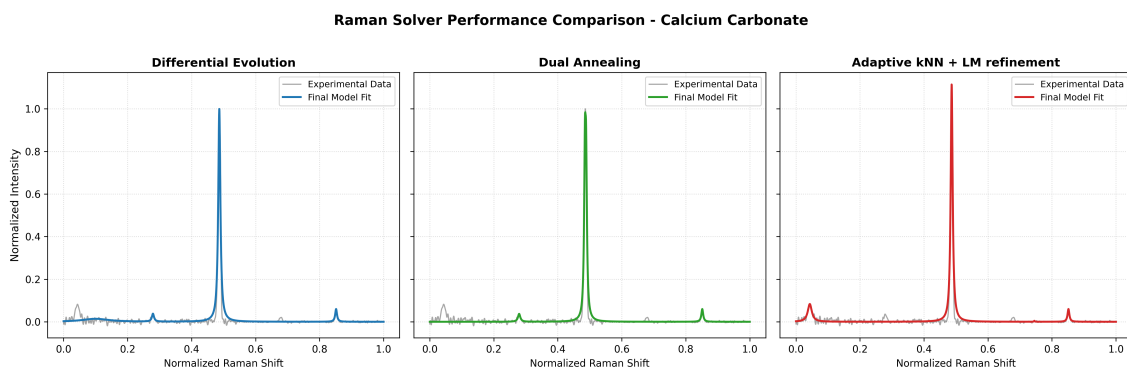


Figure 6. Final fits achieved by all the three algorithms on the Calcium Carbonate sample shown in figure 5. It is possible to see that the tested algorithms successfully converged on approximately 75% of the features annotated within the database.

Although the experimental samples within the AMES Raman Database provide exceptionally high signal quality and highly distinct spectral profiles, some specific samples, such as the amino acid Alanine, were selected to introduce a more rigorous evaluation challenge due to the high density of overlapping features within a restricted spectral window.

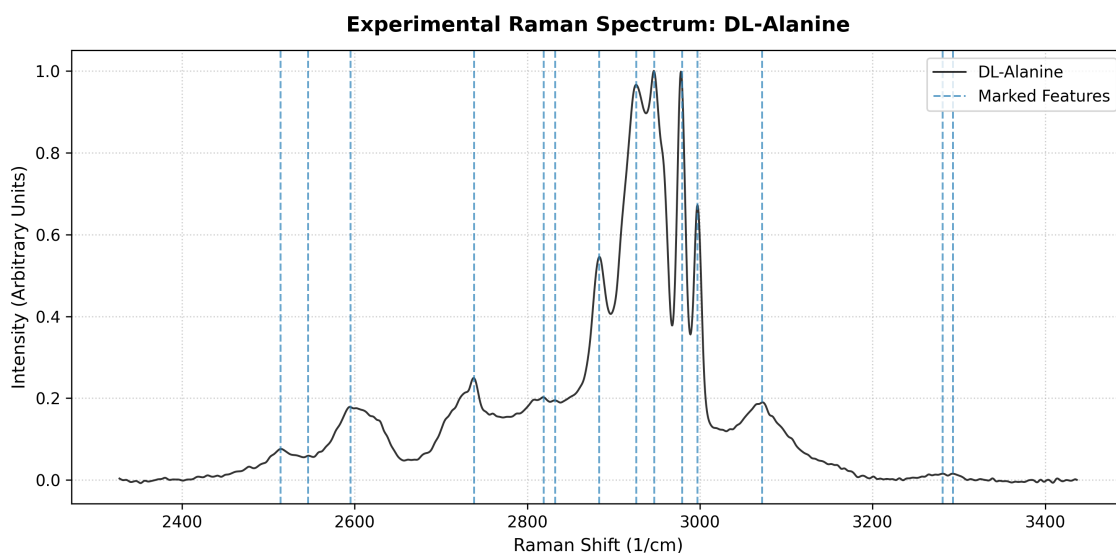


Figure 7. Shape of the spectral window between 2327 and 3437 cm^{-1} of DL-Alanine. The blue vertical lines show the positions of the features labeled by the maintainers of the database. There are 14 features in this very small spectral window. The authors haven't annotated effectively all Raman features and a lot of them are overlapping and difficult to detect, despite being a fairly clean sample in terms of white noise.

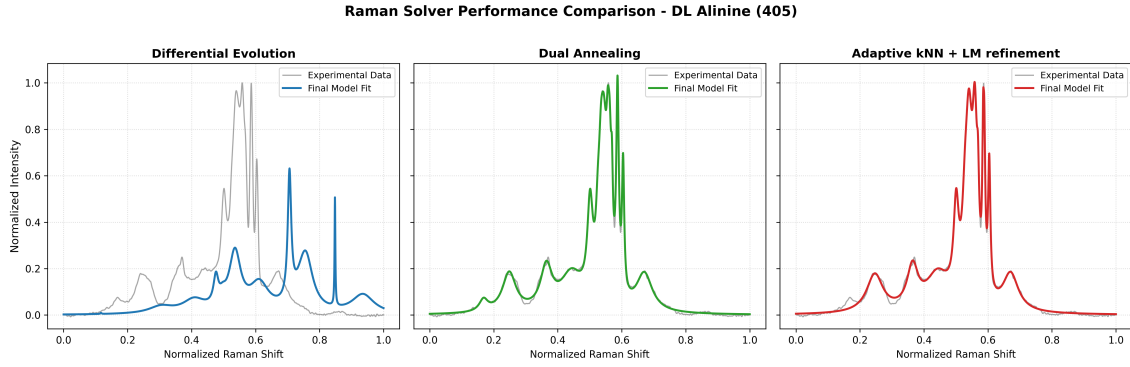


Figure 8. Output of the algorithms on the DL-Alanine sample shown in figure 7. Differential Evolution performed particularly poorly in this test. For DA and A-kNN, some of the peaks were not found and, as a consequence, the algorithms treat prominent peaks as convolutions of more than one peak at the same position.

Table 2 presents the optimization results for all experimental samples. Differential Evolution provided the most pronounced anomalies among the evaluated methods, completely failing to converge on successful fits for Magnesium Sulfate and DL-Alanine. Conversely, Dual Annealing and the proposed Adaptive kNN framework combined with Levenberg-Marquadt demonstrated comparable performance regarding fit quality across all spectral profiles.

Table 2. Features Detected per Method on Real Spectra

Sample Name and Spectral Window	Features Detected / Total		
	A-kNN + LM	DA	DE
Alanine (2327 – 3437 cm^{-1})	10/14	10/14	0/14
Calcium Carbonate (200 – 2016 cm^{-1})	3/4	3/4	3/4
Calcium Sulfate (310 – 1235 cm^{-1})	3/5	4/5	4/5
Calcium Sulfate (3237 – 3730 cm^{-1})	2/2	2/2	2/2
Dolomite (200 – 1648 cm^{-1})	2/2	2/2	2/2
Iron(II,III) Oxide (1944 – 2760 cm^{-1})	1/1	1/1	1/1
Iron(II,III) Oxide (200 – 821 cm^{-1})	4/5	4/5	4/5
Magnesium Carbonate (200 – 1960 cm^{-1})	3/5	3/5	4/5
Magnesium Sulfate (200 – 3999 cm^{-1})	6/13	6/13	0/13
Olivine (200 – 1672 cm^{-1})	5/8	5/8	3/8

It is important to emphasize that all evaluated algorithms are stochastic, and thus their outputs are non-deterministic, meaning it cannot be definitively inferred whether an algorithm will converge to a global minimum. Although these frameworks may fail to capture specific features during a single execution, they may provide higher convergence rates in successive trials. This behavior is especially relevant in the case of Differential Evolution. Furthermore, the outputs of Differential Evolution and Dual Annealing can also be refined using local optimizers, albeit at the expense of scaling the overall computational complexity.

3.3. Full Pipeline

The integrated performance of the complete pipeline was evaluated across a validation database comprising 300 synthetic Raman spectra. The sequential architecture of the framework was executed as follows: automated peak counting via the modified SLMK-CNN detailed in Figure 2, initial parameter estimation using the A-kNN algorithm, and final parameter refinement via LM and BIC model selector. The target datasets contained between one and five peaks, with stochastic noise levels applied across standard deviation values ranging from $\sigma = 0.01$ to $\sigma = 0.5$.

We used the same condition established by equation 8 to determine whether a fit was successful with a smaller error threshold $\epsilon = 10^{-6}$. A smaller ϵ means a much smaller error tolerance to judge the quality of the fit. The complete pipeline achieved a success rate of 95.33% on the 300 spectral samples. The pipeline correctly classified the exact number of peaks in 61.67% of the cases

To contextualize these experiments, four representative samples were randomly extracted and present in the Figures 9, 10, 11 e 12. The classification status (match vs mismatch), indicates whether the correct peak-count architecture was selected by the BIC framework. A mismatch does not inherently imply a sub-optimal fit, rather, it indicates that a lower-dimensional solution was found sufficient. Resolving ambiguous superpositions with fewer parameters is generally considered a preferable objective compared to fitting too many parameters to simple curves. This preference arises because executing a fit with more parameters not only introduces information without scientific significance but also artificially distorts the remaining parameters within the spectral superposition.

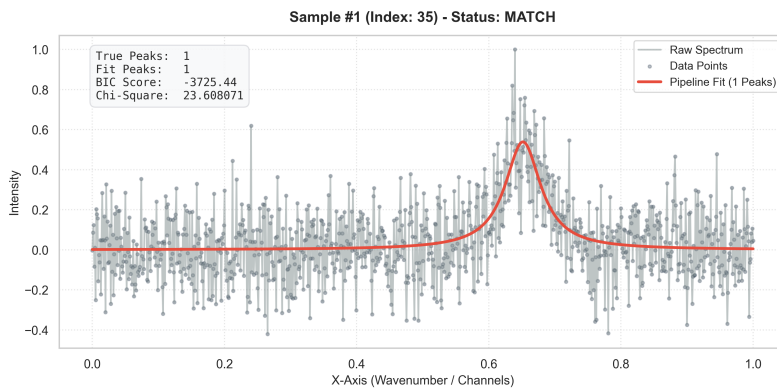


Figure 9. First random sample drawn from the test set. Noise is quite intense but the peak is still fairly visible.

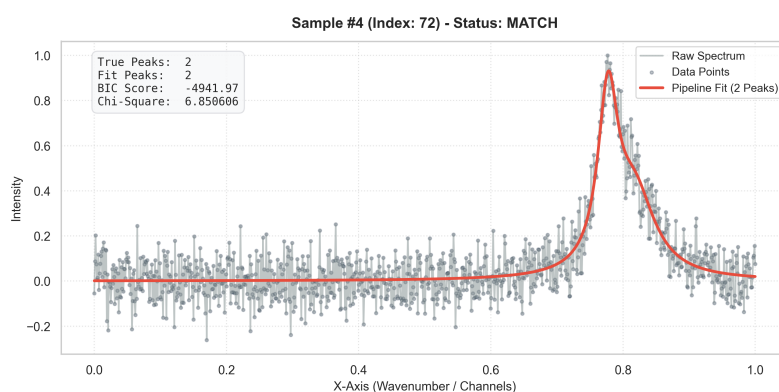


Figure 10. Second random sample drawn from the test set. This sample shows two curves on a superposition, but they are still easily distinguishable due to their difference in γ values. In this case the model chosen matched the true peak counts.

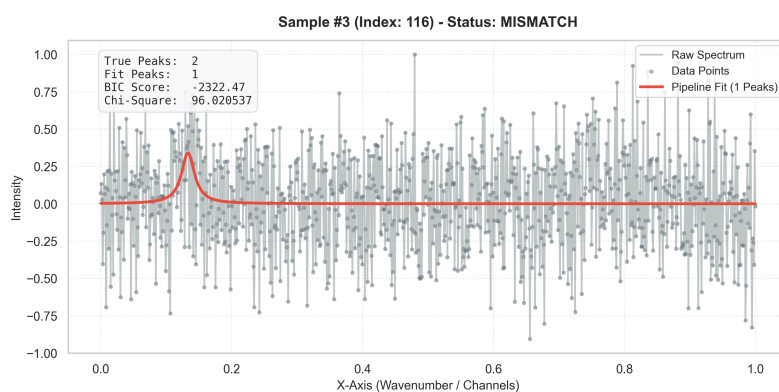


Figure 11. Third random sample drawn from the test set. Noise is very intense in this image and it is difficult to see where peaks are located. There are in fact 2 hidden peaks in this image, but the simplest model located and fit only a single peak.

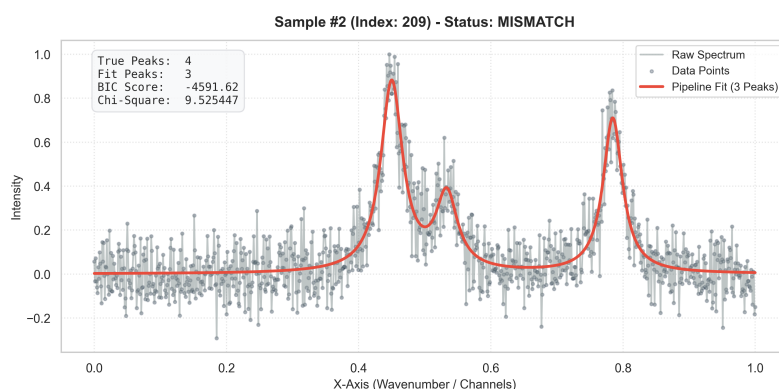


Figure 12. Fourth random sample drawn from the test set. This is an interesting case as there are 3 true peaks within the superposition between 0.4 and 0.6 in the normalized range, plus the other peak closer to the 0.8 mark. However, the simplified model only fitted 3 peaks, which in this case provided a good minimal solution to the spectrum.

The specific condition under which a spectral superposition of overlapping peaks can be successfully resolved occurs when either the γ values or the amplitudes of the two features are highly asymmetric. In these scenarios, even under significant spectral overlap, the respective curves exhibit sufficient distinctiveness to remain resolvable. However, because of the stochastic nature of the integrated frameworks, an optimal convergence still cannot be guaranteed.

To formalize the proposed pipeline as a proof of concept, validation was also conducted on selected experimental samples from the dataset evaluated in Section 3.2.1. Because the neural network was trained to estimate peak counts from spectra characterized by exactly 1,000 data points spanning one to five peaks, this rigid structural requirement represents its primary limitation. Consequently, the selected samples were downsampled or cropped to match this 1,000 points of resolution. These selected samples were Calcium Carbonate, Calcium Sulfate and Olivine, because they contained up to five annotated features or presented rigorous challenges to the pipeline. Exhaustive testing of the entire dataset was deemed unnecessary since, given an accurate estimation of peak counts from Stage I, it has already been demonstrated in Section 3.2.1 that stages II and III are fully capable of achieving optimal convergence. The results for the selected samples are shown in Figures 13, 14 and 15, where it's possible to compare the fitted peaks with the marked peaks that were expected to be fitted.

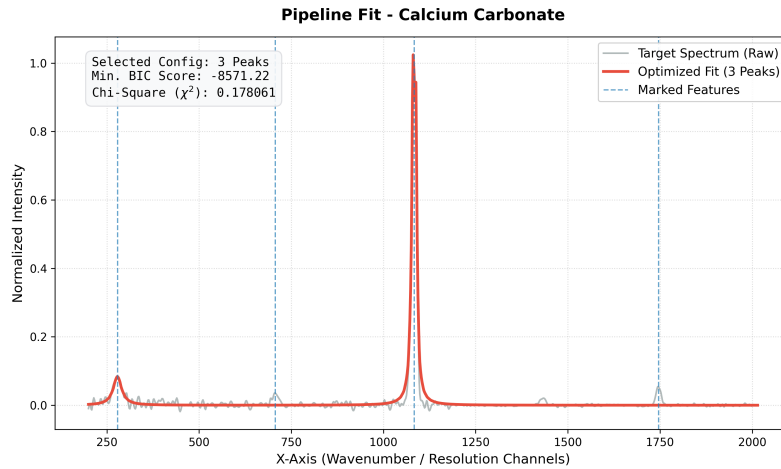


Figure 13. Calcium Carbonate evaluated by the the complete pipeline. Positions of labeled features from the database are marked as vertical lines. The pipeline fitted 2 out of 4 features, attributing the most prominent peak a sum of two peaks.

Global fitting optimizers, including the proposed A-kNN, struggle a lot to properly fit features with very small amplitude values in relation to a set of very prominent peaks. The A- kNN's probability map is designed to mitigate this limitation, but such anomalies may still persist in specific scenarios. This is because when minimizing the χ^2 , or the Euclidean distance in the case of A-kNN, prominent peaks affect the computed errors much more significantly than small peaks.

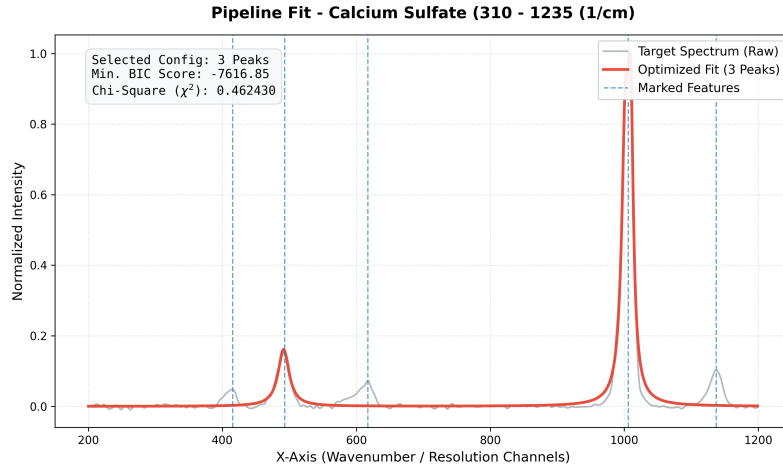


Figure 14. Calcium Sulfate evaluated by the the complete pipeline. Positions of labeled features from the database are marked as vertical lines. The database annotates 5 distinct features, but our pipeline detected 3 and fitted 2.

This limitation is also reflected within the integrated pipeline. The outcome includes anomalous cases where, despite a high SNR, the framework fails to achieve optimal convergence. Several strategies can be implemented to mitigate this limitation, which are discussed in the Future Work Section.

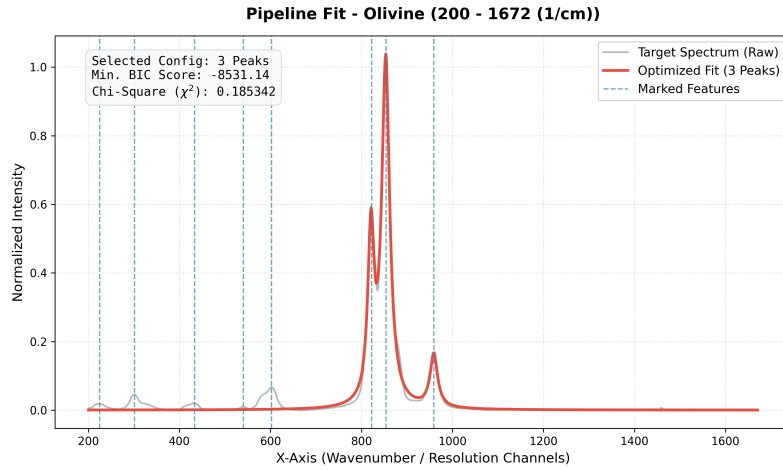


Figure 15. Olivine evaluated by the the complete pipeline. Positions of labeled features from the database are marked as vertical lines. The database annotates 8 distinct features. The proposed pipeline can detect up to 5 peaks but marked only the 3 most prominent ones.

4. Future Work

For future versions of the automated fitting pipeline, the primary objectives focus on enhancing flexibility and successfully resolving complex edge-case scenarios where automated fitting remains highly challenging yet necessary for complete framework functionality. A greedy optimization approach is highly inefficient when processing noisy

spectra or dense spectral overlaps. However, it may represent a highly effective framework to handle cases where SNR is high and peaks are easily distinguishable.

The primary limitations of a greedy approach, if deployed in isolation, involve its high noise sensitivity and the requirement for a robust stopping criterion. Algorithm 2 presents a solution where the estimated peak counts from Stage I serve as the definitive stopping criterion. Alternatively, another possible stopping criterion involves calculating the noise σ by selecting all data points below the intensity median along the y-axis) and fitting a Gaussian distribution. Usually, noise is calculated by computing the standard deviation across a flat spectral window lacking features, such approach requires manual selection of those windows.

Algorithm 2 Greedy Peak Fitting

```

1: procedure FITSPECTRUM
  Input:
     $S$  : target spectrum
     $n$  : number of peaks
  Output:
    Set of fitted peak parameters  $P$ 

2:    $R \leftarrow S$  ▷ Initialize residual spectrum
3:    $P \leftarrow \emptyset$ 
4:   for  $i = 1$  to  $n$  do
5:     Locate maximum intensity in residual spectrum  $R$ 
6:     Estimate peak width at the maximum
7:     Fit peak parameters using Levenberg–Marquardt optimization
8:     Subtract fitted peak from residual spectrum
9:     Add fitted peak parameters to  $P$ 
10:  end for
11:  return  $P$ 
12: end procedure

```

It may be possible to combine the greedy approach with A-kNN, but first it must be verified if computing the lowest Bayesian Information Criterion score over the combined results will select unsuitable models produced by the greedy approach on high noise conditions, or if a decision criterion should be established for the pipeline to choose which of the two methods is best applicable to the target spectra being analyzed.

Another possibility for future improvements to the pipeline involves increasing its flexibility around new dataset definitions. Currently, the neural network is very rigid, providing accurate results only under a specific spectral resolution and defined global parameter boundaries. Any alteration to these operational variables requires full model retraining, a process that demands several hours. Random Forest, however, can retrain using alternative dataset parameterizations within minutes, while maintaining satisfactory accuracy to remain a viable candidate architecture for future iterations of the pipeline. Additionally, an ensemble methodology with Scipy’s native peak finder algorithm is also under consideration.

Experiments also will be performed to demonstrate that some superpositions of peaks may be simply unsolvable by local or global optimizers. Despite the efficacy of the methods implemented within automated curve-fitting architectures, they cannot fully overcome the limitations of local or global optimizers, since these algorithms are utilized for the final parameter refinements. Consequently, alternative frameworks must be explored to detect severe spectral superpositions and resolve them.

5. Conclusion

An automated curve-fitting pipeline was introduced and demonstrated to reliably perform curve fitting and feature detection across an extensive number of both real and synthetic samples, significantly minimizing human-in-the-loop. This integration of machine learning with a global fitting algorithm and a model selector is capable of resolving all sequential phases involved in Raman spectral analysis. However, the methodology remains in an early development stage. The pipeline still encounters difficulties when resolving low amplitude features within spectra dominated by highly prominent peaks, even under high SNR where a successful convergence should be straightforward given the high quality of the underlying signals.

Future development will focus on addressing these limitations and enhancing framework reliability of the pipeline by combining a greedy optimization approach with the A-kNN algorithm. Furthermore, the current iteration exhibits a lack of flexibility within its machine learning component, which requires multiple hours of training overhead to recalibrate for new parameter boundaries. While the Random Forest model is considered a strong contender to replace the neural network, further exploration is yet necessary to maximize its predictive accuracy. This assessment will establish whether Random Forest is superior for this task, and whether the compromise of a minor accuracy penalty is an acceptable trade-off justified by the substantial gains in maximizing system flexibility and operational utility.

6. Generative AI Statement

The authors declare generative AI use was limited to the following purposes: as a research tool to find libraries, documentation online and books, for an initial brainstorming of ideas, and as a coding assistant. For these purposes, a combination of ChatGPT (OpenAI), Gemini (Google) and DeepSeek was used at different stages of the work. The text was edited through Overleaf, which has a built-in AI grammar checking tool. No AI system was used to generate research data, analyses, interpretations, or scientific conclusions. All content was written, reviewed, and validated by the authors, who take full responsibility for the accuracy and integrity of the work.

References

- da Silva Neto, E. G. (2017). *Resonance Raman Spectroscopy in Twisted Bilayer Graphene*. PhD thesis, Universidade Federal de Minas Gerais.
- Endres, S. C., Sandrock, C., and Focke, W. W. (2018). A simplicial homology algorithm for lipschitz optimisation. *Journal of Global Optimization*, 72(2):181–217.
- Ferrari, A. C., Meyer, J. C., Scardaci, V., Casiraghi, C., Lazzeri, M., Mauri, F., Piscanec, S., Jiang, D., Novoselov, K. S., Roth, S., et al. (2006). Raman spectrum of graphene and graphene layers. *Physical review letters*, 97(18):187401.

- Gouadec, G. and Colombari, P. (2007). Raman spectroscopy of nanostructures and nano-sized materials.
- Griffiths, D. J. (2013). *Introduction to Electrodynamics*. Pearson, Boston, MA, 4th edition.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.
- Jones, R. R., Hooper, D. C., Zhang, L., Wolverson, D., and Valev, V. K. (2019). Raman techniques: fundamentals and frontiers. *Nanoscale research letters*, 14(1):231.
- Lünsdorf, N. and Lünsdorf, J. (2014). Automated curve-fitting of raman spectra by an iterative, randomized approach applied to carbonaceous matter and minerals. In *11th International GeoRaman Conference*, volume 1783, page 5026.
- Mattioda, A., Gavilan, L., Ricketts, C., Najeeb, P., Ricca, A., and Boersma, C. (2024). The nasa raman spectroscopic database: Ramdb version 1.00. *Icarus*, 408:115769.
- Meza Ramirez, C. A., Greenop, M., Ashton, L., and Rehman, I. U. (2021). Applications of machine learning in spectroscopy. *Applied Spectroscopy Reviews*, 56(8-10):733–763.
- Mozaffari, M. H. and Tay, L.-L. (2022). Overfitting one-dimensional convolutional neural networks for raman spectra identification. *Spectrochimica Acta Part A: Molecular and Biomolecular Spectroscopy*, 272:120961.
- Newville, M., Otten, R., Nelson, A., Stensitzki, T., Ingargiola, A., Allan, D., Fox, A., Carter, F., and Rawlik, M. (2025). Lmfit: Non-linear least-squares minimization and curve-fitting for python.
- Orlando, A., Franceschini, F., Muscas, C., Pidkova, S., Bartoli, M., Rovere, M., and Tagliaferro, A. (2021). A comprehensive review on raman spectroscopy applications. *Chemosensors*, 9(9):262.
- Raman, C. V. (1922). *Molecular diffraction of light*. University of Calcutta.
- Raman, C. V. and Krishnan, K. S. (1928). A new type of secondary radiation. *Nature*, 121(3048):501–502.
- Rayleigh, L. (1899). Xxxiv. on the transmission of light through an atmosphere containing small particles in suspension, and on the origin of the blue of the sky. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 47(287):375–384.
- Righini, G. (1995). A double annealing algorithm for discrete location/allocation problems. *European Journal of Operational Research*, 86(3):452–468.
- Smith, E. and Dent, G. (2019). *Modern Raman spectroscopy: a practical approach*. John Wiley & Sons.

- Sohn, W. B., Lee, S. Y., and Kim, S. (2020). Single-layer multiple-kernel-based convolutional neural network for biological raman spectral analysis. *Journal of Raman Spectroscopy*, 51(3):414–421.
- Storn, R. and Price, K. (1997). Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11(4):341–359.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272.
- Wojdyr, M. (2010). Fityk: a general-purpose peak fitting program. *Applied Crystallography*, 43(5):1126–1128.
- Zhang, S.-L. (2012). *Raman spectroscopy and its application in nanostructures*. John Wiley & Sons.